

TÍTULO: Sistema de Gestão e Processamento de Dados para Aplicativo de Delivery — Tia Lu Food App

Autores:

Samir Santos Cabral
Marco Douglas Ramos Pereira
Igor Flôres Veloso
Felipe Miranda dos Santos
Jean Moraes Dutra Júnior

Resumo

Este trabalho apresenta o desenvolvimento de um sistema modular para gerenciamento e processamento de dados utilizado no aplicativo fictício *Tia Lu Food App*, com foco em restaurantes da região do Ceará. A solução foi construída em Python e organizada em múltiplos módulos independentes, permitindo manipulação de pedidos, armazenamento estruturado de dados, ordenação, exibição de menus e uso de uma árvore AVL para otimização de buscas. O projeto demonstra uma arquitetura clara, modularizada e escalável, capaz de atender demandas reais de organização e processamento de informações em sistemas de delivery.

Palavras-chave: Estrutura de Dados, AVL, Python, Sistemas de Delivery, Processamento de Dados.

1. Introdução

Sistemas de delivery têm se tornado cada vez mais relevantes com o crescimento do comércio digital e da demanda por serviços de alimentação online. Para garantir a eficiência desses sistemas, é necessário projetar soluções robustas e escaláveis que permitam manipular grandes quantidades de dados de forma rápida e organizada.

Este trabalho descreve a implementação de um sistema em Python voltado ao gerenciamento de informações e pedidos de estabelecimentos gastronômicos. Inspirado no aplicativo *Tia Lu Food App*, o projeto utiliza estruturas de dados adequadas, como árvores AVL, além de módulos específicos para manipulação de menus, pedidos, armazenamento e

ordenação. O objetivo é apresentar uma arquitetura clara, modular e funcional, capaz de demonstrar conceitos importantes da computação aplicada a sistemas reais.

2. Fundamentação Teórica

O desenvolvimento do sistema utiliza conceitos fundamentais de estruturas de dados, processamento de informações e organização modular de software.

2.1 Árvores AVL

A árvore AVL, presente no arquivo *avl.py*, é uma estrutura de dados binária auto-balanceada que garante tempo de busca, inserção e remoção em $O(\log n)$. O uso dessa estrutura permite consultas mais rápidas sobre dados armazenados, sendo ideal para sistemas com grande volume de informações.

2.2 Armazenamento de Dados

O arquivo *dados.json* é responsável por armazenar as informações utilizadas pelo aplicativo. O módulo *storage.py* faz a leitura e manipulação desses dados de maneira estruturada, permitindo fácil integração com módulos internos.

2.3 Módulos e Programação Modular

A aplicação foi organizada em componentes independentes: menu, pedidos, ordenação, árvore AVL e leitura de dados. Essa abordagem facilita a manutenção, reuso de código e expansão futura do sistema.

3. Metodologia

O sistema foi estruturado seguindo o paradigma modular, dividindo responsabilidades de forma clara:

- **main.py**: módulo controlador da aplicação, responsável pela execução geral.
- **menu.py**: controla a lógica de exibição e manipulação do menu.
- **ordenacao.py**: implementa métodos de ordenação para organizar listas de dados.
- **pedido.py**: responsável por criar e gerenciar pedidos.
- **storage.py**: faz a conexão entre o sistema e a base de dados *dados.json*.

- **avl.py**: contém a implementação da árvore AVL, aumentando performance em buscas.

Todos os módulos conversam entre si de forma organizada, permitindo fluxo coerente de informações.

4. Desenvolvimento do Sistema

A seguir, apresenta-se um resumo do papel de cada componente:

4.1 avl.py

Implementa estrutura AVL completa: nós, balanceamento, rotações e inserção. Seu uso permite armazenamento otimizado e acesso rápido a informações.

4.2 menu.py

Gerencia o menu do aplicativo, lista opções, organiza itens e apresenta funcionalidades para interação com o usuário.

4.3 ordenacao.py

Fornece algoritmos de ordenação personalizados para organizar listas, como restaurantes, pratos ou pedidos.

4.4 pedido.py

Define a classe *Pedido*, contendo métodos para criação, registro, identificação e manipulação de pedidos.

4.5 storage.py

Carrega os dados do arquivo *dados.json* e fornece funções para acessar, manipular e atualizar essas informações.

4.6 main.py

É o ponto de entrada da aplicação, responsável por inicializar módulos e orquestrar a execução completa do sistema.

5. Resultados

A aplicação, quando executada, demonstra:

- Organização eficiente de dados gastronômicos do Ceará.
- Estrutura modular clara, facilitando compreensão do código.
- Uso eficiente da árvore AVL para consultas mais rápidas.
- Facilidade na manipulação de pedidos e menus.
- Integração entre leitura de dados, ordenação e estruturas avançadas.

O sistema se mostra funcional, escalável e pronto para adaptações ou expansões para um aplicativo real.

6. Conclusão

O desenvolvimento do sistema *Tia Lu Food App* demonstrou a importância da modularidade e das estruturas de dados avançadas, como árvores AVL, para melhorar o desempenho e organização de aplicações reais. A abordagem do projeto permite fácil manutenção, integração e expansão futura, servindo como um exemplo claro de aplicação prática de conceitos fundamentais de Ciência da Computação.

O trabalho atingiu seus objetivos, construindo uma solução completa, funcional e bem estruturada para gerenciamento de dados em sistemas de delivery.

Referências

- Adelson-Velskii, G. M.; Landis, E. M. "An algorithm for the organization of information." 1962.
- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Algoritmos – Teoria e Prática*. MIT Press.
- Python Software Foundation. *Documentation*.
- Material disponibilizado pelo professor referente ao projeto.